```
/*

Async patterns

Callbacks Promises Async/Await Generators

*/

function delayPromise() { return new Promise((resolve, reject) => { console.log("from delay promise");
//resolve() }); }

function greeting() { console.log("good morning"); }

delayPromise().then(greeting); // greetning is not executed because we did not resolve a promise

//we can almost add setTimeout function delayPromise(second) { return new Promise((resolve, reject) => {
console.log("from delay promise"); setTimeout(() => { resolve(); }, second); }); }

function greeting() { console.log("good morning"); }

delayPromise(2000).then(greeting);

//we can resolve with a value function delayPromise(second) { return new Promise((resolve, reject) => {
console.log("from delay promise"); setTimeout(() => { resolve("from resolve"); }, second); }); }

function greeting(data) { console.log(data); }

delayPromise(2000).then((data) => { greeting(data); });

//We can also use reject instead of resolve function delayPromise(seconds, isTrue) { return new
Promise((resolve, reject) => { console.log("from delay promise"); setTimeout(() => { if (isTrue) { resolve("from
resolve"); } else { reject("error"); } }, seconds); }); }

function greeting(data) { console.log(data); }

delayPromise(2000, false) .then((data) => { greeting(data); }) .catch((err) => { console.log(err); });

//chaining promise - use this in the browser new Promise((resolve, reject) => { setTimeout(() => resolve(1),
2000); }) .then((data) => { alert(data); return data + 1; }) .then((data) => { alert(data); return data + 1; })
.then((data) => { alert(data); return data + 1; }) .catch((e) => { console.log(e); });
```